

Also show that a polynomial number of calls to polynomial-time subroutines may result in an exponential-time algorithm.

34.1-6

Show that the class P , viewed as a set of languages, is closed under union, intersection, concatenation, complement, and Kleene star. That is, if $L_1, L_2 \in P$, then $L_1 \cup L_2 \in P$, $L_1 \cap L_2 \in P$, $L_1 L_2 \in P$, $\bar{L}_1 \in P$, and $L_1^* \in P$.

34.2 Polynomial-time verification

Now, let's look at algorithms that verify membership in languages. For example, suppose that for a given instance $\langle G, u, v, k \rangle$ of the decision problem PATH, you are also given a path p from u to v . You can check whether p is a path in G and whether the length of p is at most k , and if so, you can view p as a “certificate” that the instance indeed belongs to PATH. For the decision problem PATH, this certificate doesn't seem to buy much. After all, PATH belongs to P —in fact, you can solve PATH in linear time—and so verifying membership from a given certificate takes as long as solving the problem from scratch. Instead, let's examine a problem for which we know of no polynomial-time decision algorithm and yet, given a certificate, verification is easy.

Hamiltonian cycles

The problem of finding a hamiltonian cycle in an undirected graph has been studied for over a hundred years. Formally, a *hamiltonian cycle* of an undirected graph $G = (V, E)$ is a simple cycle that contains each vertex in V . A graph that contains a hamiltonian cycle is said to be *hamiltonian*, and otherwise, it is *nonhamiltonian*. The name honors W. R. Hamilton, who described a mathematical game on the dodecahedron (Figure 34.2(a)) in which one player sticks five pins in any five consecutive vertices and the other player must complete the path to form a cycle containing all the vertices.⁸ The dodecahedron is hamiltonian, and Figure 34.2(a) shows one hamiltonian cycle. Not all

graphs are hamiltonian, however. For example, Figure 34.2(b) shows a bipartite graph with an odd number of vertices. Exercise 34.2-2 asks you to show that all such graphs are nonhamiltonian.

Here is how to define the *hamiltonian-cycle problem*, “Does a graph G have a hamiltonian cycle?” as a formal language:

$\text{HAM-CYCLE} = \{\langle G \rangle : G \text{ is a hamiltonian graph}\}.$

How might an algorithm decide the language HAM-CYCLE? Given a problem instance $\langle G \rangle$, one possible decision algorithm lists all permutations of the vertices of G and then checks each permutation to see whether it is a hamiltonian cycle. What is the running time of this algorithm? It depends on the encoding of the graph G . Let’s say that G is encoded as its adjacency matrix. If the adjacency matrix contains n entries, so that the length of the encoding of G equals n , then the number m of vertices in the graph is $\Omega(\sqrt{n})$. There are $m!$ possible permutations of the vertices, and therefore the running time is $\Omega(m!) = \Omega(\sqrt{n}!) = \Omega(2^{\sqrt{n}})$, which is not $O(n^k)$ for any constant k . Thus, this naive algorithm does not run in polynomial time. In fact, the hamiltonian-cycle problem is NP-complete, as we’ll prove in Section 34.5.

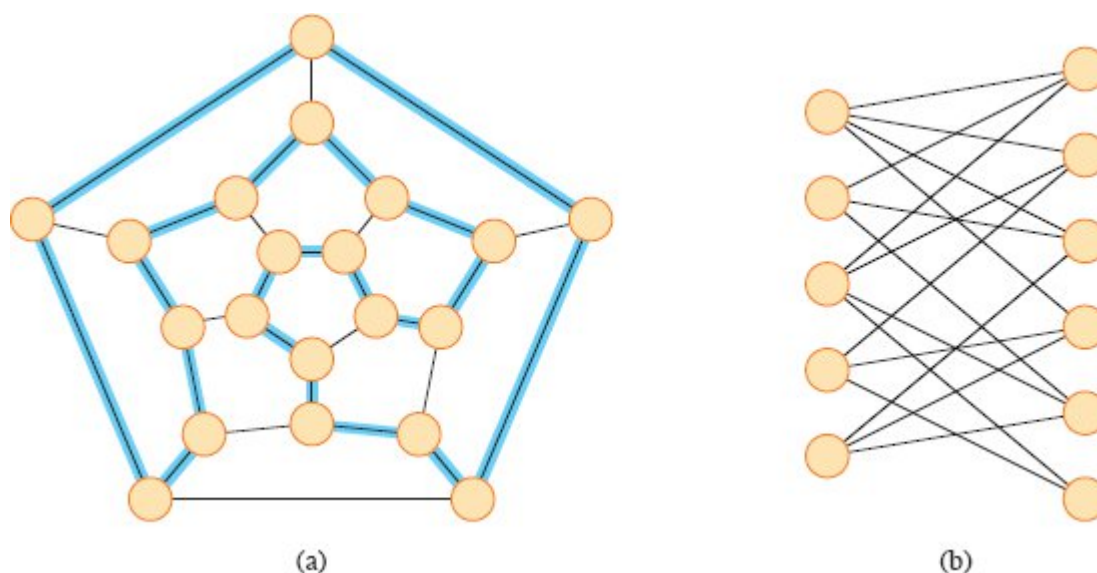


Figure 34.2 (a) A graph representing the vertices, edges, and faces of a dodecahedron, with a hamiltonian cycle shown by edges highlighted in blue. (b) A bipartite graph with an odd number of vertices. Any such graph is nonhamiltonian.

Verification algorithms

Consider a slightly easier problem. Suppose that a friend tells you that a given graph G is hamiltonian, and then the friend offers to prove it by giving you the vertices in order along the hamiltonian cycle. It would certainly be easy enough to verify the proof: simply verify that the provided cycle is hamiltonian by checking whether it is a permutation of the vertices of V and whether each of the consecutive edges along the cycle actually exists in the graph. You could certainly implement this verification algorithm to run in $O(n^2)$ time, where n is the length of the encoding of G . Thus, a proof that a hamiltonian cycle exists in a graph can be verified in polynomial time.

We define a **verification algorithm** as being a two-argument algorithm A , where one argument is an ordinary input string x and the other is a binary string y called a **certificate**. A two-argument algorithm A **verifies** an input string x if there exists a certificate y such that $A(x, y) = 1$. The **language verified** by a verification algorithm A is

$$L = \{x \in \{0, 1\}^* : \text{there exists } y \in \{0, 1\}^* \text{ such that } A(x, y) = 1\}.$$

Think of an algorithm A as verifying a language L if, for any string $x \in L$, there exists a certificate y that A can use to prove that $x \in L$. Moreover, for any string $x \notin L$, there must be no certificate proving that $x \in L$. For example, in the hamiltonian-cycle problem, the certificate is the list of vertices in some hamiltonian cycle. If a graph is hamiltonian, the hamiltonian cycle itself offers enough information to verify that the graph is indeed hamiltonian. Conversely, if a graph is not hamiltonian, there can be no list of vertices that fools the verification algorithm into believing that the graph is hamiltonian, since the verification algorithm carefully checks the so-called cycle to be sure.

The complexity class NP

The **complexity class NP** is the class of languages that can be verified by a polynomial-time algorithm.⁹ More precisely, a language L belongs to NP if and only if there exist a two-input polynomial-time algorithm A and a constant c such that

$$L = \{x \in \{0, 1\}^*: \text{there exists a certificate } y \text{ with } |y| = O(|x|^c) \\ \text{such that } A(x, y) = 1\}.$$

We say that algorithm A **verifies** language L **in polynomial time**.

From our earlier discussion about the hamiltonian-cycle problem, you can see that HAM-CYCLE $\in$ NP. (It is always nice to know that an important set is nonempty.) Moreover, if $L \in P$, then $L \in$ NP, since if there is a polynomial-time algorithm to decide L , the algorithm can be converted to a two-argument verification algorithm that simply ignores any certificate and accepts exactly those input strings it determines to belong to L . Thus, $P \subseteq$ NP.

That leaves the question of whether $P =$ NP. A definitive answer is unknown, but most researchers believe that P and NP are not the same class. Think of the class P as consisting of problems that can be solved quickly and the class NP as consisting of problems for which a solution can be verified quickly. You may have learned from experience that it is often more difficult to solve a problem from scratch than to verify a clearly presented solution, especially when working under time

constraints. Theoretical computer scientists generally believe that this analogy extends to the classes P and NP , and thus that NP includes languages that do not belong to P .

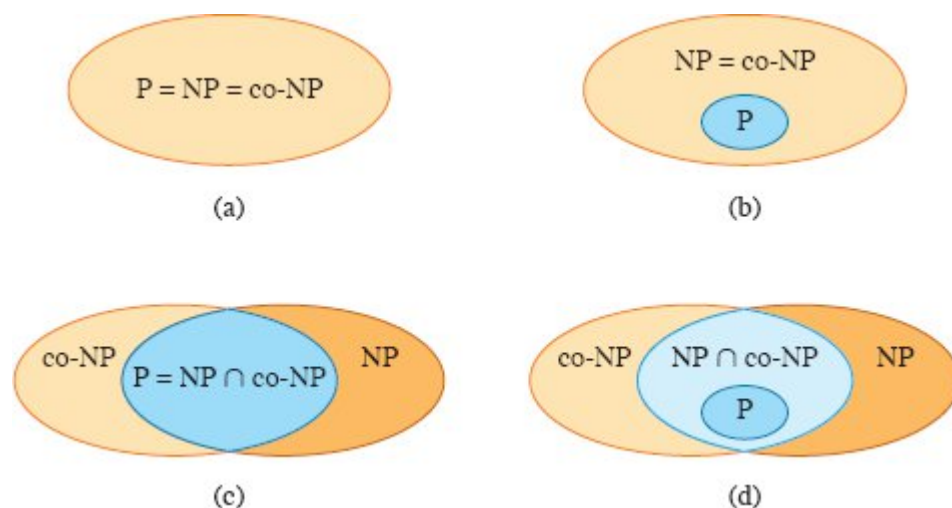


Figure 34.3 Four possibilities for relationships among complexity classes. In each diagram, one region enclosing another indicates a proper-subset relation. **(a)** $P = NP = co-NP$. Most researchers regard this possibility as the most unlikely. **(b)** If NP is closed under complement, then $NP = co-NP$, but it need not be the case that $P = NP$. **(c)** $P = NP \cap co-NP$, but NP is not closed under complement. **(d)** $NP \neq co-NP$ and $P \neq NP \cap co-NP$. Most researchers regard this possibility as the most likely.

There is more compelling, though not conclusive, evidence that $P \neq NP$ —the existence of languages that are “NP-complete.” Section 34.3 will study this class.

Many other fundamental questions beyond the $P \neq NP$ question remain unresolved. Figure 34.3 shows some possible scenarios. Despite much work by many researchers, no one even knows whether the class NP is closed under complement. That is, does $L \in NP$ imply $\bar{L} \in NP$? We define the **complexity class co-NP** as the set of languages L such that $\bar{L} \in NP$, so that the question of whether NP is closed under complement is also whether $NP = co-NP$. Since P is closed under complement (Exercise 34.1-6), it follows from Exercise 34.2-9 ($P \subseteq co-NP$) that $P \subseteq NP \cap co-NP$. Once again, however, no one knows whether $P = NP \cap co-NP$ or whether there is some language in $(NP \cap co-NP) - P$.

Thus our understanding of the precise relationship between P and NP is woefully incomplete. Nevertheless, even though we might not be able to prove that a particular problem is intractable, if we can prove that it is NP-complete, then we have gained valuable information about it.

Exercises

34.2-1

Consider the language GRAPH-ISOMORPHISM = $\{\langle G_1, G_2 \rangle : G_1 \text{ and } G_2 \text{ are isomorphic graphs}\}$. Prove that GRAPH-ISOMORPHISM $\in$ NP by describing a polynomial-time algorithm to verify the language.

34.2-2

Prove that if G is an undirected bipartite graph with an odd number of vertices, then G is nonhamiltonian.

34.2-3

Show that if HAM-CYCLE $\in$ P, then the problem of listing the vertices of a hamiltonian cycle, in order, is polynomial-time solvable.

34.2-4

Prove that the class NP of languages is closed under union, intersection, concatenation, and Kleene star. Discuss the closure of NP under complement.

34.2-5

Show that any language in NP can be decided by an algorithm with a running time of $2^{O(n^k)}$ for some constant k .

34.2-6

A *hamiltonian path* in a graph is a simple path that visits every vertex exactly once. Show that the language HAM-PATH = $\{\langle G, u, v \rangle : \text{there is a hamiltonian path from } u \text{ to } v \text{ in graph } G\}$ belongs to NP.

34.2-7

Show that the hamiltonian-path problem from Exercise 34.2-6 can be solved in polynomial time on directed acyclic graphs. Give an efficient algorithm for the problem.

34.2-8

Let ϕ be a boolean formula constructed from the boolean input variables $x_1, x_2, \dots, x_k$, negations ($\neg$), ANDs ($\wedge$), ORs ($\vee$), and parentheses. The formula ϕ is a *tautology* if it evaluates to 1 for every assignment of 1 and 0 to the input variables. Define TAUTOLOGY as the language of boolean formulas that are tautologies. Show that TAUTOLOGY $\in$ co-NP.

34.2-9

Prove that $P \subseteq \text{co-NP}$.

34.2-10

Prove that if $\text{NP} \neq \text{co-NP}$, then $P \neq \text{NP}$.

34.2-11

Let G be a connected, undirected graph with at least three vertices, and let G^3 be the graph obtained by connecting all pairs of vertices that are connected by a path in G of length at most 3. Prove that G^3 is hamiltonian. (*Hint*: Construct a spanning tree for G , and use an inductive argument.)

34.3 NP-completeness and reducibility

Perhaps the most compelling reason why theoretical computer scientists believe that $P \neq \text{NP}$ comes from the existence of the class of NP-complete problems. This class has the intriguing property that if *any* NP-complete problem can be solved in polynomial time, then *every* problem in NP has a polynomial-time solution, that is, $P = \text{NP}$. Despite decades of study, though, no polynomial-time algorithm has ever been discovered for any NP-complete problem.